

AI CONCEPTS

A Developer's Field Guide

Everything you need to know about LLMs, Embeddings, RAG, Vector Databases, Prompt Engineering, and AI Agents — explained simply for backend engineers.

LLMs	Tokens	Embeddings	RAG	Vector DB	Prompts	Chunking	Agents
------	--------	------------	-----	-----------	---------	----------	--------

For: Backend Engineers learning AI | **Stack:** Language-agnostic concepts | **Level:** Beginner to Intermediate

Table of Contents

01 Large Language Models (LLMs)	What LLMs are, tokens, context window, temperature, hallucinations
02 Embeddings	Vectors, dimensions, cosine similarity, semantic meaning
03 Chunking	Why we chunk, strategies, chunk size and overlap
04 Vector Databases	Storage, indexing, similarity search, ANN
05 RAG — Retrieval-Augmented Generation	Full pipeline, ingestion, retrieval, generation
06 Prompt Engineering	System prompts, user prompts, few-shot, chain-of-thought
07 LLM APIs	REST structure, OpenAI compatibility, Groq, free tiers
08 AI Agents	What makes an agent, tools, memory, planning, ReAct

CHAPTER 01

Large Language Models (LLMs)

What is an LLM?

A Large Language Model (LLM) is a type of AI that has been trained on massive amounts of text data — books, websites, code, articles — to understand and generate human language. Think of it as a very sophisticated autocomplete. Given some text input, it predicts what should come next, token by token.

Modern LLMs are built on the Transformer architecture (introduced in the 'Attention Is All You Need' paper, 2017). They have billions of parameters that encode patterns learned during training. Popular LLMs include GPT-4, Llama 3, Claude, and Gemini.

Tokens — The Unit of Language

LLMs do not read text character by character or word by word. They read **tokens**. A token is roughly 0.75 words on average. The word 'unhappy' might be split into 'un' and 'happy' — two tokens. Numbers, punctuation, and spaces are also tokens.

Text	Tokens	Count
Hello World	'Hello' + ' World'	2
I am unhappy today	'I' + ' am' + ' un' + 'happy' + ' today'	5
spring.ai.openai.api-key	'spring' + '.' + 'ai' + '.' + 'open' + 'ai' + '.' + 'api' + '-key'	9

Why tokens matter:

- LLMs have a **token limit per request** — you can only send so much text at once.
- Pricing for paid LLM APIs is calculated per token.
- Chunking strategies in RAG are designed around token limits.

Context Window

The context window is the maximum number of tokens an LLM can 'see' at one time — both your input (prompt) and its output (response) together. Think of it as the LLM's working memory. If your prompt + document + answer exceeds the context window, the LLM literally cannot process it.

Model	Context Window	Approx. Pages
Llama 3 70B (Groq)	8,192 tokens	~6 pages
GPT-4o	128,000 tokens	~96 pages
Claude 3.5	200,000 tokens	~150 pages
Gemini 1.5 Pro	1,000,000 tokens	~750 pages

Temperature — Controlling Creativity

Temperature is a number (usually 0.0 to 1.0) that controls how random or creative the LLM's output is. This is a parameter you set when calling the LLM API.

- **Temperature 0.0** — Deterministic, always gives the most probable answer. Best for factual Q&A; and RAG.
- **Temperature 0.5** — Balanced. Some creativity but still grounded.
- **Temperature 1.0** — Creative and varied. Good for writing, brainstorming.

Key Insight: For your PDF RAG agent, always use temperature 0.0 or 0.1. You want accurate, factual answers from the document — not creative hallucinations.

Hallucinations

When an LLM generates text that sounds confident but is factually wrong or completely made up, that is called a hallucination. LLMs don't 'know' facts — they predict statistically likely next tokens. This is the core problem that RAG solves: instead of letting the LLM guess from training data, you give it the actual relevant text from your document as context.

CHAPTER 02

Embeddings

What is an Embedding?

An embedding is a way of converting text (or any data) into a list of numbers — called a **vector**. These numbers capture the **meaning** of the text in a mathematical space. Text with similar meaning gets similar vectors. This is the foundation of semantic search.

Example: The words 'king', 'queen', 'prince' are all close to each other in vector space because they are semantically related. 'Database' and 'table' are close to each other. 'Cat' and 'automobile' are far apart.

The Famous Word Analogy:

$$\text{vector('king')} - \text{vector('man')} + \text{vector('woman')} \approx \text{vector('queen')}$$

This works because embeddings encode relationships, not just surface words. The model learned that 'king' is to 'man' as 'queen' is to 'woman' — purely from patterns in training text.

Dimensions

A vector's dimension is how many numbers it contains. Modern embedding models produce vectors with 384 to 1536 dimensions. Each dimension captures some abstract aspect of meaning — though no single dimension has a simple human label. Higher dimensions generally capture more nuance.

Example: A 4-dimensional simplified embedding of 'cat' might look like:

```
cat → [0.82, -0.14, 0.67, 0.31, ..., 0.09] (384 numbers total)
```

Cosine Similarity

Once text is converted to vectors, we need a way to measure how similar two vectors are. The standard method is **cosine similarity** — it measures the angle between two vectors in high-dimensional space.

Score	Meaning	Example
1.0	Identical meaning	'car' vs 'car'

0.85+	Very similar	'automobile' vs 'vehicle'
0.5 - 0.8	Related	'spring boot' vs 'java framework'
0.0 - 0.3	Unrelated	'database' vs 'ocean'
-1.0	Opposite meaning	'hot' vs 'cold'

Key Insight: When you ask a question to your RAG app, that question gets embedded into a vector. Then cosine similarity finds which stored document chunks are closest in meaning — not by keyword matching, but by semantic meaning.

Chunking

Why Do We Chunk?

You cannot embed an entire 100-page PDF as one vector. Two reasons: first, LLMs have token limits — you cannot send 100 pages as context. Second, embedding a huge document produces one blurry vector that captures nothing specifically. You need focused, small chunks so each embedding captures one clear idea.

Think of it like an index in a book. Instead of searching the whole book, you search the index (vector store) to find exactly which page (chunk) contains what you need.

Chunking Strategy

- **Chunk Size:** How many tokens per chunk. Typical range: 256 to 1024 tokens. Smaller chunks = more precise retrieval. Larger chunks = more context per result.
- **Overlap:** How many tokens are shared between consecutive chunks. Typical: 10-20% of chunk size. Overlap prevents losing information at chunk boundaries.
- **Splitting method:** Split by sentence, paragraph, or fixed token count. Sentence-aware splitting preserves meaning better than hard cuts.

Visual Example — Chunk with Overlap:

Original Text (30 tokens): | Token 1 ... Token 10 | Token 11 ... Token 20 | Token 21 ... Token 30 |
Chunk 1 (size=15, overlap=5): | Token 1 Token 15 |
Chunk 2 (starts at token 11): | Token 11 Token 25 |
Chunk 3 (starts at token 21): | Token 21 ... Token 30 |

Recommended Settings for PDF RAG:

Parameter	Value	Reason
Chunk Size	500 tokens	One idea per chunk, fits most context windows
Overlap	50 tokens (10%)	Prevents losing sentences at boundaries
Split On	Sentence boundary	Preserves full thoughts, better embeddings

CHAPTER 04

Vector Databases

What is a Vector Database?

A vector database is a specialized database that stores embeddings (vectors) and lets you search them by similarity — not by exact match. Traditional databases like MySQL search by exact value (WHERE name = 'John'). A vector DB searches by meaning: 'find me the 5 chunks most semantically similar to this question.'

How It Works:

Store: Each document chunk is embedded → vector stored alongside original text and metadata.

Index: Vectors are indexed using ANN (Approximate Nearest Neighbor) algorithms for fast search.

Query: Your question is embedded → compared against all stored vectors using cosine similarity.

Return: Top-K most similar chunks are returned with their similarity scores.

Popular Vector Databases:

Database	Type	Best For	Free?
SimpleVectorStore	In-memory	Learning, prototyping (Spring AI built-in)	Yes
ChromaDB	Local / Docker	Development, persistent storage	Yes
Weaviate	Cloud / Self-hosted	Production, free tier available	Free tier
Qdrant	Cloud / Self-hosted	High performance, open source	Free tier
pgvector	PostgreSQL extension	Already using Postgres, production	Yes
Pinecone	Managed cloud	Production at scale	Paid

For your PDF RAG project: Start with SimpleVectorStore (zero setup, built into Spring AI). Once it works end-to-end, upgrade to ChromaDB via Docker for persistence between restarts.

CHAPTER 05

RAG — Retrieval-Augmented Generation

What is RAG?

RAG is an architecture pattern that makes LLMs more accurate and trustworthy by giving them relevant external information at query time, rather than relying purely on what they memorized during training. It directly solves hallucination for domain-specific questions.

Without RAG: you ask the LLM a question → it guesses from training data → may hallucinate. With RAG: you first retrieve relevant documents → stuff them into the prompt → LLM answers using real context.

The Two Phases of RAG:

Phase 1 — Ingestion (done once when you upload a document)

1. Load the document (PDF, DOCX, text etc.)
2. Extract raw text from the document
3. Split text into chunks (e.g. 500 tokens with 50 overlap)
4. Embed each chunk using an embedding model → vector
5. Store each vector + original chunk text in the vector database

Phase 2 — Retrieval + Generation (done every time user asks a question)

1. Receive user's question
2. Embed the question using the SAME embedding model
3. Run similarity search in vector store → get top-K matching chunks
4. Build a prompt: system instruction + retrieved chunks + user question
5. Send prompt to LLM
6. Return LLM's answer to user

The RAG Prompt Structure:

```
SYSTEM: You are a helpful assistant. Answer ONLY based on the context provided. If the answer is not in the context, say 'I don't know'. CONTEXT (retrieved chunks from vector store): [Chunk 1 text here...] [Chunk 2 text here...] [Chunk 3 text here...]  
USER: What are the key concepts explained in this document?
```

Why RAG Works Better Than Just Sending the Full Document:

Problem	Without RAG	With RAG
Large documents	Can't fit in context window	Only relevant chunks sent
Accuracy	LLM guesses from training	Answers from actual document
Cost	Huge token usage	Minimal tokens per query
Speed	Slow (large context)	Fast (small focused context)

Prompt Engineering

What is Prompt Engineering?

Prompt engineering is the practice of crafting the input you send to an LLM to get the best possible output. It is not about magic words — it is about giving the LLM clear instructions, context, and constraints. As a backend engineer building a RAG app, you will write prompts programmatically in code.

Anatomy of a Prompt:

- System Prompt:** Sets the LLM's role and rules. 'You are a helpful assistant that answers only from the provided context.'
- Context:** The retrieved chunks from your vector store inserted into the prompt dynamically.
- User Message:** The actual question the end user asked.
- Examples:** Optional: show the LLM a sample input/output to guide its response format (few-shot).
- Output Format:** Tell the LLM how to structure its answer: JSON, bullet points, paragraph, etc.

Key Prompt Techniques:

Zero-shot	Give no examples. Just instruct the LLM. Works for simple tasks.
Few-shot	Give 2-3 examples of input/output before the real question. Dramatically improves accuracy for specific formats.
Chain-of-Thought	Ask the LLM to 'think step by step'. Improves reasoning on complex questions.
Role Prompting	Set a persona: 'You are a senior Java architect...'. Changes tone and depth.
Constrained Output	Tell LLM to respond ONLY in JSON, or to answer in 3 bullet points, etc.

Good Prompt vs Bad Prompt for RAG:

Bad Prompt	Good Prompt
------------	-------------

LLM APIs

How LLM APIs Work

Almost every LLM provider exposes their model through a simple REST API. You send a POST request with your prompt, and get a response with the LLM's answer. The most important thing to know: most providers are compatible with OpenAI's API format, meaning the same client code works across providers — you just change the base URL and model name.

Standard Request Structure:

```
POST https://api.groq.com/openai/v1/chat/completions Headers: Authorization: Bearer YOUR_API_KEY Content-Type: application/json Body: { "model": "llama3-70b-8192", "temperature": 0.1, "messages": [ { "role": "system", "content": "You are a helpful assistant..." }, { "role": "user", "content": "What is RAG?" } ] }
```

Free LLM API Options:

Provider	Free Tier	Best Model	Speed
Groq	Generous daily limits	Llama 3 70B	Very fast
Google Gemini	Free tier available	Gemini 1.5 Flash	Fast
Ollama (local)	Completely free	Llama 3, Mistral	Depends on hardware
HuggingFace	Free inference API	Many open models	Variable
OpenRouter	Free models available	Multiple providers	Variable

Spring AI Configuration for Groq (Free):

```
# application.properties spring.ai.openai.base-url=https://api.groq.com/openai spring.ai.openai.api-key=${GROQ_API_KEY} spring.ai.openai.chat.options.model=llama3-70b-8192 spring.ai.openai.chat.options.temperature=0.1
```

Spring AI reuses the OpenAI client library — you just point the base URL at Groq. Zero code changes, only configuration changes. This is the power of OpenAI-compatible APIs.

AI Agents

What is an AI Agent?

An AI Agent is an LLM that can take actions — not just answer questions. A regular chatbot receives a question and generates an answer. An agent can decide what tools to use, call external APIs, search the web, run code, read files, and chain multiple steps together to complete a complex goal.

Chatbot vs Agent:

Regular LLM / Chatbot	AI Agent
Answers questions from memory or context	Plans steps, uses tools, takes actions
Single input → single output	Multi-step reasoning loop
Passive — waits for prompt	Active — decides what to do next
No external access	Can call APIs, search web, run code
Example: Q&A; chatbot	Example: 'Book me a flight for Friday' agent

Core Components of an Agent:

- LLM (Brain):** The reasoning engine. Decides what to do, interprets results, forms responses.
- Tools:** Functions the agent can call: search web, read file, call API, run SQL query, etc.
- Memory:** Short-term (conversation history in context window) and long-term (vector store).
- Planning:** The agent breaks a goal into steps and executes them in sequence.
- Observation:** The agent reads tool results and decides what to do next based on them.

ReAct Pattern — How Agents Think:

The most popular agent pattern is ReAct (Reason + Act). The agent loops through: Think about the goal → Choose a tool to use → Use the tool → Observe the result → Think again → Repeat until goal is reached.

```
Goal: 'Summarize the key findings in the uploaded PDF' Thought 1: I need to read the PDF first. Action 1: search_vector_store('key findings') Observation 1: [returns top 5 relevant chunks] Thought 2: I have enough context to summarize now. Action 2: generate_summary(chunks) Observation 2: Summary generated. Final Answer: Here are the key findings...
```

Your PDF RAG App is Actually an Agent:

The Spring AI app you are building — where the LLM uses a vector store tool to retrieve relevant chunks before answering — is already an agent pattern. The `QuestionAnswerAdvisor` in Spring AI implements exactly this: retrieve context using the vector store tool, then generate an answer using that context.

Summary of all concepts: LLM reads tokens → Embedding converts text to vectors → Chunks split the document → Vector DB stores and searches chunks → RAG retrieves relevant chunks → Prompt engineering combines them → LLM API generates the answer → Agent orchestrates the whole flow automatically.